

Representação e Armazenamento de Informação Digital

Laboratórios de Sistemas e Serviços

Mário Antunes

Universidade de Aveiro

2026

Introdução

Na era digital, estamos rodeados de dados, mas o que é que eles significam?

- **Dados:** Factos e números em bruto sem contexto (ex: "38.5").
- **Informação:** Dados que foram processados, organizados ou estruturados para serem significativos (ex: "A temperatura do paciente é 38.5°C").
- **Conhecimento:** A capacidade de usar a informação para tomar decisões.
- **Sabedoria:** O uso integrado do conhecimento.

Para transformar dados em informação, precisamos de:

- **Estrutura:** Um formato predefinido para os dados.
- **Contexto:** Informação sobre o que os dados representam.
- **Metadados:** “Dados sobre dados” (ex: unidades, carimbos temporais, origem).

Um modelo para representar as relações estruturais e funcionais entre dados e sabedoria.

- **Dados:** A fundação (símbolos).
- **Informação:** Dados ligados (responde a quem, o quê, onde, quando).
- **Conhecimento:** Informação aplicada (responde a como).
- **Sabedoria:** Conhecimento avaliado (responde a porquê).

Porquê a Representação Importa I

A forma como representamos os dados afeta todas as fases do ciclo de vida:

- **Armazenamento:** Quanto espaço ocupa (compressão).
- **Velocidade:** Quão rápido podemos ler ou escrever os dados.
- **Interoperabilidade:** Conseguem sistemas diferentes entender os mesmos dados?
- **Leitura Humana:** Pode um humano depurar ou editar o ficheiro facilmente?

Porquê a Representação Importa II

- **Escalabilidade:** O formato funciona para 1GB? 1TB?
- **Segurança:** Os dados podem ser facilmente adulterados?
- **Longevidade:** O formato será legível daqui a 20 anos?
- **Validação:** Podemos verificar se os dados estão corretos?

Categorias de Datos

Os dados são geralmente classificados em três categorias baseadas na sua estrutura:

1. **Dados Não Estruturados:** Sem formato predefinido.
2. **Dados Semi-Estruturados:** Têm algumas propriedades organizacionais mas sem esquema rígido.
3. **Dados Estruturados:** Seguem um modelo rigoroso e predefinido (geralmente tabular).

- **Escolha:** A categoria depende da natureza dos dados e de como serão usados.
- **Migração:** Frequentemente extraímos dados estruturados de fontes não estruturadas (ex: processar logs).
- **Ferramentas:** Cada categoria requer ferramentas e competências diferentes.

Dados Não Estruturados

Características de Dados Não Estruturados

Dados não estruturados são o tipo mais comum de dados no mundo.

- **Exemplos:** Documentos de texto, ficheiros PDF, emails, imagens, vídeos, logs.
- **Formato:** Geralmente binário ou texto simples sem uma estrutura interna consistente.
- **Desafio:** É difícil de pesquisar, indexar e analisar usando ferramentas tradicionais.
- **Crescimento:** Estimado em 80% de todos os dados empresariais.

Processamento de Dados Não Estruturados: Ferramentas Básicas I

Antes de usar ferramentas avançadas, usamos utilitários Unix básicos para explorar texto:

- `head / tail`: Ver o início ou o fim de um ficheiro.
- `cat / less`: Imprimir ou navegar no conteúdo do ficheiro.
- `wc`: Contar linhas, palavras e caracteres.
- `file`: Identificar o tipo de dados (ex: texto, imagem, binário).

Processamento de Dados Não Estruturados: Ferramentas Básicas II

Exemplo de exploração:

```
$ file data.log  
$ wc -l data.log  
$ head -n 5 data.log
```

Expressões Regulares (Regex) I

Para processar texto não estruturado, precisamos de uma forma de descrever padrões.

- **Literal:** abc corresponde a “abc”.
- **Wildcard:** . corresponde a qualquer carácter.
- **Quantificadores:**
 - *: 0 ou mais.
 - +: 1 ou mais.
 - ?: 0 ou 1.
 - {n,m}: entre n e m.

- **Âncoras:**

- `^`: Início da linha.
- `$`: Fim da linha.

- **Classes de Caracteres:**

- `[a-z]`: Qualquer letra minúscula.
- `\d`: Qualquer dígito.
- `\w`: Qualquer carácter de palavra (letras, números, underscore).
- `\s`: Qualquer espaço em branco.

Exemplo de Regex: Validação de Email

Um padrão simplificado: `^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`

- `^[a-zA-Z0-9._%+-]+`: Um ou mais caracteres no início.
- `@`: O símbolo literal @.
- `[a-zA-Z0-9.-]+`: O nome do domínio.
- `\.`: Um ponto literal.
- `[a-zA-Z]{2,}$`: O domínio de topo (pelo menos 2 letras) no fim.

Processamento de Dados Não Estruturados

Os logs são um exemplo clássico de dados “quase” não estruturados.

- Cada ação num sistema gera uma entrada de log.
- Frequentemente, são apenas linhas de texto num ficheiro.
- **Ferramentas:** Usamos “power tools” de Unix para extrair informação.
 - grep: Pesquisar padrões.
 - awk: Processar colunas de texto.
 - sed: Editor de fluxo para transformação de texto.

Processamento de Dados Não Estruturados: Logs II

Exemplo de um Apache Access Log: 127.0.0.1 - -
[20/Apr/2026:09:24:00 +0000] "GET /index.html
HTTP/1.1" 200 612

- Parece estruturado, mas é apenas uma string.
- Para saber qual o IP que mais visitou, precisamos de processar o texto.
- `cat access.log | awk '{print $1}' | sort | uniq -c | sort -nr`

Processamento Avançado de Logs: awk

O awk é uma linguagem de programação completa para processamento de texto.

- Processa ficheiros linha a linha, divididos em campos (\$1, \$2, ...).
- **Uso:** `awk '$9 == 404 {print $7}' access.log`
- Este comando imprime o URL (\$7) para todos os pedidos que resultaram num erro 404 (\$9).

O sed é usado para transformar ou filtrar texto.

- **Substituição:** `sed 's/velho/novo/g' ficheiro.txt`
- **Eliminação:** `sed '1,5d' ficheiro.txt` (Eliminar linhas 1 a 5).
- **Extração:** `sed -n 's/.*ID:\([0-9]*\).*/\1/p' log.txt`
- O sed é extremamente rápido e pode processar ficheiros enormes que não cabem na memória.

Dados Semi-Estruturados

O que são Dados Semi-Estruturados? I

Não residem numa tabela fixa mas contêm etiquetas ou marcadores para separar os elementos de dados.

- **Flexibilidade:** Pode representar facilmente relações hierárquicas e aninhadas.
- **Auto-Descritivo:** As etiquetas fornecem metadados dentro do próprio ficheiro.
- **Formatos Comuns:** JSON, XML, YAML.

O que são Dados Semi-Estruturados? II

- **Uso:**

- APIs Web (REST/GraphQL).
- Ficheiros de configuração.
- Bases de dados NoSQL (MongoDB, CouchDB).
- Troca de dados entre sistemas heterogéneos.

JSON (JavaScript Object Notation) I

O padrão “de facto” para comunicação web moderna.

- **Formato:** Pares Chave-Valor e Arrays.
- **Prós:** Leve, fácil de ler para humanos, fácil de processar para máquinas.
- **Tipos de Dados:** String, Número, Booleano, Nulo, Objeto, Array.

JSON (JavaScript Object Notation) II: Tipos de Dados

- **String:** "mario"
- **Número:** 123, 12.3
- **Booleano:** true, false
- **Nulo:** null
- **Objeto:** {"chave": "valor"}
- **Array:** [1, 2, 3]

JSON (JavaScript Object Notation) III: Exemplo

```
{  
  "user": "mario",  
  "id": 123,  
  "active": true,  
  "roles": ["admin", "teacher"],  
  "address": {  
    "city": "Aveiro",  
    "zip": "3810"  
  }  
}
```

Para garantir que um ficheiro JSON está correto, usamos um **JSON Schema**.

- Define a estrutura, campos obrigatórios e tipos de dados.
- Permite a validação automática antes do processamento.

```
{  
  "type": "object",  
  "properties": {  
    "user": {"type": "string"},  
    "id": {"type": "integer", "minimum": 1}  
  },  
  "required": ["user", "id"]  
}
```

XML (eXtensible Markup Language) I

Um padrão mais antigo e verboso focado na estrutura de documentos.

- **Formato:** Etiquetas aninhadas
`<tag>conteúdo</tag>`.
- **Prós:** Extremamente robusto, suporta esquemas complexos (XSD), muito rigoroso.
- **Contras:** “Pesado” (muito overhead de metadados), mais difícil de ler que JSON.

XML (eXtensible Markup Language) II: Atributos

Ao contrário do JSON, o XML pode armazenar dados em **atributos**.

```
<user id="123" status="active">  
  <name>mario</name>  
</user>
```

- **Etiquetas:** Boas para dados hierárquicos.
- **Atributos:** Bons para metadados sobre uma etiqueta.

XML (eXtensible Markup Language) III: Esquema (XSD)

- **XSD (XML Schema Definition):** Uma forma de definir as regras para um ficheiro XML.
- Define quais as etiquetas permitidas, a sua ordem e o tipo de dados que contêm.
- Isto permite a **validação automática** dos dados.

XML (eXtensible Markup Language) IV: Namespaces

O XML usa **Namespaces** para evitar colisões de etiquetas ao combinar documentos diferentes.

```
<root xmlns:h="http://www.w3.org/TR/html4/"  
      xmlns:f="https://www.w3schools.com/furniture">  
  <h:table>...</h:table>  
  <f:table>...</f:table>  
</root>
```

- h:table refere-se a uma tabela HTML.
- f:table refere-se a uma tabela de mobiliário.

YAML (YAML Ain't Markup Language) I

Desenhado para ser o formato de dados mais amigável para humanos.

- **Formato:** Usa indentação em vez de chavetas ou etiquetas.
- **Prós:** Muito limpo, fácil de escrever, suporta comentários.
- **Uso:** Ficheiros de configuração (Docker, Kubernetes, GitHub Actions).
- **Aviso:** A indentação é crítica (como em Python).

YAML (YAML Ain't Markup Language) II: Funcionalidades

- **Listas:** Começadas com um traço -.
- **Comentários:** Usar #.
- **Strings multi-linha:** Usar | (manter quebras de linha) ou > (agrupar quebras de linha).

- **Âncoras e Aliases:** Reutilizar dados dentro do mesmo ficheiro.

```
defaults: &base  
  adapter: postgres  
  host: localhost
```

```
development:  
  <<: *base  
  database: dev_db
```

YAML (YAML Ain't Markup Language) III: Exemplo

```
user: mario
id: 123
active: true
roles:
  - admin
  - teacher
address:
  city: Aveiro
  zip: 3810
```

Dados Estruturados

Dados que encaixam perfeitamente numa tabela (linhas e colunas).

- **Esquema:** Cada linha deve ter as mesmas colunas.
- **Eficiência:** Muito rápido de pesquisar e processar usando SQL ou dataframes.
- **Exemplos:** CSV, TSV, Bases de Dados Relacionais (SQL).

CSV (Comma Separated Values) I

O formato mais simples e comum para troca de dados tabulares.

- **Formato:** Cada linha é um registo; campos são separados por uma vírgula (,).
- **Prós:** Suporte universal (Excel, Python, R, Bases de Dados).
- **Contras:** Sem forma padrão de lidar com caracteres especiais ou dados aninhados.

CSV (Comma Separated Values) II: O “Problema”

E se um campo contiver uma vírgula? 1,Mario
Antunes,"Aveiro, Portugal",true

- Campos com vírgulas ou aspas devem ser **protegidos com aspas**.
- Diferentes países usam diferentes delimitadores (ex: ; em vez de ,).
- **TSV (Tab Separated Values)**: Usa tabs para evitar o problema da vírgula.

Os ficheiros CSV sofrem frequentemente de problemas de codificação.

- **UTF-8:** O padrão moderno (suporta todas as línguas).
- **ISO-8859-1 (Latin-1):** Comum em ficheiros antigos de Windows/Excel.
- **Problema:** Abrir um ficheiro Latin-1 como UTF-8 resulta em “mojibake” (ex: Ãj em vez de á).

Formatos Binários Estruturados (Brevemente)

Para Big Data, o CSV é demasiado lento e grande.

- **Apache Parquet:** Um formato de armazenamento colunar. Eficiente para ler colunas específicas.
- **Apache Avro:** Um formato baseado em linhas com um esquema. Ótimo para fluxos de dados.
- **Prós:** Compressão, segurança de tipos, significativamente mais rápido que texto.

Exploração de Dados

jq é como o sed para dados JSON. É essencial para engenheiros de CLI.

- Permite filtrar, transformar e formatar JSON a partir da linha de comandos.
- **Uso:** `cat data.json | jq '.user'`
- **Formatação:** `cat data.json | jq '.'` (Prettify).

```
cat users.json | jq '.[ ] | select(.active == true) | .name'
```

1. `.[ ]`: Iterar sobre o array.
2. `select(...)`: Filtrar baseado numa condição.
3. `.name`: Extrair apenas o campo do nome.

yq é a versão YAML do jq.

- Frequentemente usado para editar ficheiros de configuração programaticamente.
- `yq eval '.server.port = 8081' config.yml`
- Essencial para pipelines CI/CD (GitHub Actions).

Dados em Python I: CSV

```
import csv

# Leitura
with open('data.csv', 'r', encoding='utf-8') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['name'], row['age'])

# Escrita
with open('out.csv', 'w', newline='', encoding='utf-8') as f:
    writer = csv.writer(f)
    writer.writerow(['id', 'name'])
    writer.writerow([1, 'mario'])
```


Dados em Python II: JSON

```
import json

# Parsing de uma string
obj = json.loads('{"id": 1, "name": "mario"}')

# Guardar num ficheiro
with open('data.json', 'w') as f:
    json.dump(obj, f, indent=4)
```

Dados em Python III: YAML

```
import yaml # Requer PyYAML

# Carregar
with open('config.yml', 'r') as f:
    config = yaml.safe_load(f)

# Exportar
print(yaml.dump(config))
```

Serialização vs Deserialização I

- **Serialização:** Converter um objeto em memória (ex: um dicionário Python) num formato que possa ser armazenado ou transmitido (ex: uma string JSON).
- **Deserialização:** O processo inverso: converter uma string ou ficheiro de volta num objeto em memória.

Serialização vs Deserialização II

Porque precisamos disto?

- **Persistência:** Guardar o estado de um programa no disco.
- **Transmissão:** Enviar um objeto pela rede (API).
- **Independência de Linguagem:** Um programa Python pode enviar um JSON para um programa Java.

Validação de Dados

Porquê Validar Dados?

Dados maus levam a resultados maus (“Garbage In, Garbage Out”).

- **Tipos:** É um número? É uma data?
- **Intervalos:** A idade está entre 0 e 120?
- **Obrigatoriedade:** O campo do email está presente?
- **Relações:** O ID do departamento existe?

Pydantic é a forma moderna de validar dados em Python.

```
from pydantic import BaseModel, EmailStr, Field
```

```
class User(BaseModel):  
    id: int  
    name: str = Field(min_length=3)  
    email: EmailStr  
    age: int = Field(gt=0, lt=120)
```

Validação com Pydantic II

```
# Dados válidos
u = User(id=1, name="Mario", email="mario@ua.pt", age=30)

# Dados inválidos (lança ValidationError)
try:
    u2 = User(id=1, name="Ma", email="not-an-email", age=-5)
except Exception as e:
    print(e)
```

- O Pydantic converte automaticamente os tipos onde possível (ex: "123" para 123).

Sumário

- **Categorias:** Não estruturados (logs), semi-estruturados (JSON/YAML), estruturados (CSV).
- **Padrões:** Usar Regex para processamento de texto e awk/sed para logs.
- **Escolha:** JSON para APIs, YAML para configuração e CSV para dados em massa.
- **Ferramentas:** Dominar jq para JSON e yq para YAML.
- **Programático:** Usar as bibliotecas padrão de Python e **Pydantic** para uma manipulação de dados robusta.
- **Validação:** Validar sempre os dados no ponto de entrada do sistema.